

P1130R1

Module Resource Requirement

Propagation

Published Proposal, 2019-01-21

Author:

[JeanHeyd Meneide](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:

https://thephd.github.io/vendor/future_cxx/papers/d1130.html

Abstract

This paper attempts to provide a level of indirection upon which build system and package management tools build sane, higher-level abstractions.

Table of Contents

1 Revision History

- 1.1 Revision 1 - January 21st, 2019
- 1.2 Revision 0 - November 26th, 2018

2 Motivation

3 Proposed Solution

4 Proposed Changes to the Standard

- 4.1 Proposed Feature Test Macro

4.2 Intent

4.3 Proposed Wording

5 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 1 - January 21st, 2019

- Add words of power, "resource location".
- Clarify wording around it not being a hard error.
- Track changes to [\[p1103\]](#) accordingly.
- Fix description in motivation of the use of `require`.
- Add HRESOURCE example.

§ 1.2. Revision 0 - November 26th, 2018

- Initial release.

§ 2. Motivation

The only way to declare a dependency in C++ currently is to use a `#include` statement. With Modules coming to C++, dependency information was greatly enhanced by preambles and globule module fragments which allow for both the compiler and build system to understand physical and semantic layout of code. However, there is still a problem area that C++ has not addressed that people in the brave new Modules ecosystem want to answer: external dependency information for Modular C++. There is a huge opportunity to add a small directive to C++ which can be transiently ignored by the compiler but allows a build system or dependency graph generator to stay up to date without compiler-specific and tool-specific hacks.

In particular, consider a resource file on Windows (`.rc`) or an injected resource on Linux (with `objcopy`) or Apple's Bundles. At present, there's no way to inform C++ of these dependencies or

inform the build system in a way that keeps both the source code and the build system in-sync: it is easy to fall out of lock-step with each other, and often requires custom rules on the part of the build system vendor or the application author. While resource files are a problem that [\[p1040\]](#) plan to solve, San Diego discussion around such a proposal demonstrated that people were conflicted with the idea that there might not be a way to communicate without doing full Semantic Analysis the actual source dependencies.

Reducing the complexity of the build system and its dependency on vendor-specific extensions and tools for handling source dependency information is of high priority. This proposal wants to add 2 new statements to the Preprocessing Module Tokens for [\[p1103\]](#), in particular `requires { identifier = requires "blah.txt" }` and `requires "blah.txt"` in the Module Declaration.

§ 3. Proposed Solution

The proposed solution to the problem is to add a general purpose marker for communication of dependencies in the preamble. There is also an addition for making the string literal used to show dependencies as a global variable, allowing it to be used in `constexpr` contexts. It will be used as part of the typical modules declaration, making it available in public and private modules. For example:

```
module;
// communicate this module
// relies on a file "foo.txt"
module bar requires "foo.txt";
```

For multiple dependencies for a single module, we use a multi-clause requires with brackets:

```
module;
// communicate this module
// relies on multiple files / assets
export module mega_bar requires { "qux.txt", "meow.jpg", "uuids.csv" };
```

This allows us to inform the build system of dependencies with no new keywords. This also lets the compiler export dependency information with user-added information at no additional cost to the compiler, not occurring semantic analysis. Note that there is no error if the compiler cannot find the file: it will be up to the build system to determine that information.

A program may also set one or more of the string literals to an identifier made available after the preamble. The value of that identifier will be the string literal set equal to the identifier in the module requires:

```
module bar requires { bar_ico_name = L"bar.ico" };

// later, same module unit ...

// using, for example,
// Windows's RC system
HRSRC hResource = FindResourceExW(m_hmoduleInstance,
    bar_ico_name,
    m_hresource_type,
    MAKELANGID(LANG_NEUTRAL, SUBLANG_NEUTRAL));
```

This allows individuals to not suffer from the multi-location-updating problem: specify the name in the preamble, use that name everywhere else without having to update multiple places or keep 2 names in sync.

The proposal's wording also introduces a "Word of Power" — *resource-locations* — to specify and allow the rest of the C++ Standard to all reference the same place about resource locations and lookup. It is important that this is distinct from inclusion paths, because the two lookup specifications are inherently separate and serve different purposes. Having it as something that can be referenced means that we can keep all resource interaction attempts -- from [p1040 - std::embed](#) to any upcoming initiatives later in the standard -- perfectly in-sync with one another.

§ 4. Proposed Changes to the Standard

These changes are relative to [\[p1103\]](#). If this is successful in being merged to C++20, then this paper will be rebased on the C++ Working Draft.

§ 4.1. Proposed Feature Test Macro

The proposed feature test macro for this is `__cpp_module_dependency_requires`.

§ 4.2. Intent

The intent of this wording is to:

- provide a Word of Power called "*resource-locations*" in the Standard that allow other places to reference the same lookup mechanism;
- allow a `requires` clause at the end of a module declaration to specify resource identifiers;
- and, allow resource identifiers to be set to a variable name that can be used throughout the *module-unit*.

This proposal wants to ensure that future and concurrent proposals (such as `std::embed`) can refer to this same lookup mechanism by using the specified word of power ("*resource-locations*").

§ 4.3. Proposed Wording

Modify section §100.1 Modules units and purview [**module.unit**] to also include the following:

module-declaration:

```
exportopt module module-name module-partitionopt attribute-specifier-seqopt module-  
requiresopt ;
```

Add a section §100.❖ Resource requirement propagation [**module.requires**] to §100 [**modules**]:

module-requires-name:string-literalidentifier = string-literal**module-requires-name-seq:**module-requires-namemodule-requires-name , module-requires-name-seq_{opt}**module-requires:**requires { module-requires-name-seq_{opt} }.requires string-literal

¹ A resource requirement is a way for a module to communicate its dependency on other resources for the program. A program may specify one or more string-literals in the requirements clause to make clear this dependency in the module-declaration.

² When the module-requires-name specifies an identifier set to a value, then the module-requires-name's identifier is made available to the module unit as-if by `decltype(string-literal)& identifier = string-literal;`.

³ The locations the implementation may search for the resource uniquely identified by the string-literal is implementation defined, and is called the resource-locations.

⁴ This information does not affect the well-formedness of the program. [Note: the implementation may choose to issue a warning if it cannot find the resource specified. — end note]

§ 5. Acknowledgements

Thanks to Isabella Muerte for helping to select the right type of syntax for this feature.

§ References

§ Informative References

[P1040]

JeanHeyd Meneide. std::embed. October 12th, 2018. URL: <https://wg21.link/p1040>

[P1103]

Richard Smith. [Merging Modules](https://wg21.link/p1103). November 26th, 2018. URL: <https://wg21.link/p1103>